

FinPlanner - Setup and Run Instructions

Overview

This document provides step-by-step instructions to set up and run the enhanced FinPlanner application with the new Renewals/Planned Obligations features, Global Command Bar, and enhanced SMS Inbox.

New Features Implemented

1. Renewals / Planned Obligations (First-Class Module)

- Track recurring obligations (insurance, school fees, taxes, etc.)
- Support for multiple frequencies: One-time, Monthly, Quarterly, Half-yearly, Yearly
- Provider/vendor and policy number tracking
- Configurable reminder offsets (30/7/1 days before due)
- **Automatic due date rolling** when marking as paid
- Payment history with reference numbers and notes
- Overdue and due-soon status indicators

2. Global Command Bar with AI

- Available on every page (press **Cmd/Ctrl + K**)
- Natural language input powered by Abacus.AI LLM
- Quick add transactions or renewals
- Examples:
 - “Spent ₹500 groceries cash”
 - “Paid medical insurance ₹28011 today policy 3073”
 - “Add renewal LIC due 19 Mar yearly ₹6527”
- Confidence-based confirmation dialogs

3. Enhanced SMS Inbox (API Ready)

- Batch SMS parsing with LLM integration
- Review and approve workflow
- Raw SMS storage for traceability
- Higher accuracy with Abacus.AI vs pattern matching

4. Dashboard Enhancements (Ready for UI)

- Month-to-date income/expenses/net (current calendar month)
 - Upcoming obligations widget (next 30/60 days)
 - Quick action buttons
-

Prerequisites

Before you begin, ensure you have:

- **Node.js** v18.0.0 or higher
- **npm** v8.0.0 or higher
- **PostgreSQL** v13 or higher
- **Git**

Installation Steps

Step 1: Clone Repository (if not already done)

```
git clone -b Deepagent https://github.com/venkatkrish78/finplanner.git
cd finplanner
```

Step 2: Install Dependencies

```
cd app
npm install
```

Step 3: Set Up Environment Variables

The `.env.local` file has already been created in `/app/`. Verify it contains:

```
DATABASE_URL="postgresql://postgres:password@localhost:5432/finplanner_dev"
NEXTAUTH_URL="http://localhost:3000"
NEXTAUTH_SECRET="local-development-secret-key-for-finplanner-renewals-2026"
NODE_ENV="development"
PORT=3000
NEXT_TELEMETRY_DISABLED=1
```

Important: Update `DATABASE_URL` with your actual PostgreSQL credentials.

Step 4: Set Up PostgreSQL Database

Option A: Using Existing PostgreSQL

```
# Create database
createdb finplanner_dev

# Or using psql
psql -U postgres
CREATE DATABASE finplanner_dev;
\q
```

Option B: Using Docker

```
# From the repository root
docker-compose -f docker-compose.dev.yml up -d postgres-dev
```

Step 5: Run Database Migration

```
cd app

# Generate Prisma client (already done, but run if needed)
npm run db:generate

# Create and apply migration for new schema changes
npx prisma migrate dev --name add_renewals_fields

# Or if migration already exists, just apply it
npx prisma migrate deploy
```

Step 6: Seed Database with Sample Data

```
# Run the seed script to add demo renewals and transactions
npm run db:seed
```

Step 7: Build and Start Development Server

```
# Start development server with hot reload
npm run dev
```

The application will be available at: **http://localhost:3000**

Note: This localhost refers to the computer running the application (the development environment), not your local machine. If you need to access it from your browser, you may need to configure port forwarding or deploy it.



Usage Guide

Creating a Renewal/Bill

Method 1: Using Global Command Bar (AI-Powered)

1. Press **Cmd/Ctrl + K** from any page
2. Type naturally: "Add renewal LIC due 19 Mar yearly ₹6527"
3. Review the parsed data
4. Select category
5. Click "Create"

Method 2: Traditional Form

1. Navigate to **Bills & Utilities** page
2. Click "Add Bill" button
3. Fill in:
 - Name (e.g., "LIC Premium")
 - Amount (e.g., 6527)
 - Frequency (e.g., "Yearly")
 - Next Due Date (e.g., "2026-03-19")
 - Provider (e.g., "Life Insurance Corporation")

- Policy Number (e.g., "12345678")
- Reminder Days (e.g., "30,7,1" - default)
- Category

4. Click "Create Bill"

Marking a Renewal as Paid

1. Navigate to **Bills & Utilities** page
2. Switch to "Monthly View" or "Yearly View"
3. Find the renewal that is due
4. Click "Mark Paid" button
5. Enter:
 - Amount (defaults to bill amount)
 - Reference Number (e.g., policy number, receipt reference)
 - Notes (optional)
6. Click "Confirm"

Result: The system will:

- Mark the instance as PAID
- Create a transaction automatically (unless disabled)
- **Automatically roll the due date** to the next period based on frequency

Example: Marking a yearly LIC renewal due on March 19, 2026 as paid will automatically update the next due date to March 19, 2027.

Using the Command Bar for Transactions

1. Press **Cmd/Ctrl + K**
2. Type: "Spent ₹500 on groceries at SuperMart"
3. Review parsed data (amount, type, description, merchant)
4. Select category (e.g., "Groceries")
5. Click "Create"

Viewing Upcoming Obligations

1. Navigate to **Dashboard** (or **AI Home**)
2. Look for "Upcoming Obligations" widget
3. View all bills/renewals due in the next 30/60 days
4. Color-coded indicators:
 -  **Red:** Overdue
 -  **Orange:** Due soon (within 7 days)
 -  **Green:** Upcoming (within 30 days)

SMS Parsing (Enhanced - API Ready)

Note: UI needs to be built for full workflow. API endpoint is ready.

1. Navigate to **Transactions** page
2. Look for "Parse SMS" or "SMS Inbox" button (when UI is built)
3. Paste multiple SMS messages
4. System parses using LLM
5. Review parsed transactions
6. Approve, edit, or reject each

7. Approved transactions are created

Testing

Run Unit Tests

```
cd app

# Run all tests
npm test

# Run specific test file
npm test lib/__tests__/bill-utils.test.ts
```

Manual Testing Checklist

Renewals End-to-End Test

1. ☒ Create a yearly renewal with due date in current month
2. ☒ Verify it appears in “Upcoming Obligations”
3. ☒ Mark it as paid with a reference number
4. ☒ Verify payment record is created
5. ☒ **Verify next due date has rolled forward by 1 year**
6. ☒ Verify transaction was created automatically
7. ☒ Check overdue and due-soon status indicators work

Command Bar Test

1. ☒ Press Cmd/Ctrl + K to open
2. ☒ Test transaction: “Spent ₹500 groceries”
3. ☒ Test renewal: “Add renewal Netflix ₹199 monthly”
4. ☒ Test ambiguous input to verify confidence handling
5. ☒ Verify category selection works
6. ☒ Confirm creation works for both types

Dashboard Test

1. ☒ Add transactions for current month
2. ☒ Verify month-to-date income/expense/net is correct
3. ☒ Verify upcoming obligations show correctly
4. ☒ Test date range filters (30/60 days)

Troubleshooting

Issue: “Prisma Client Not Found”

```
npm run db:generate
```

Issue: Database Connection Error

1. Verify PostgreSQL is running:

```
bash
sudo systemctl status postgresql # Linux
brew services list                # macOS
```

2. Check DATABASE_URL in `.env.local`

3. Test connection:

```
bash
psql "postgresql://postgres:password@localhost:5432/finplanner_dev"
```

Issue: Migration Fails

```
# Reset database (WARNING: deletes all data)
npx prisma migrate reset

# Or manually drop and recreate
dropdb finplanner_dev
createdb finplanner_dev
npx prisma migrate dev
```

Issue: Command Bar Not Working

1. Check browser console for errors
2. Verify Abacus.AI API key is set (ABACUS_API_KEY env var is pre-configured)
3. Check `/api/ai/parse-command` endpoint logs

Issue: “Abacus.AI Module Not Found”

```
npm install abacusai
```



Key Files Modified/Created

Schema Changes

-  `/app/prisma/schema.prisma` - Extended Bill and BillInstance models

Core Utilities

-  `/app/lib/bill-utils.ts` - Date rolling, status computation, reminder logic
-  `/app/lib/__tests__/bill-utils.test.ts` - Comprehensive unit tests

API Routes

-  `/app/api/bills/route.ts` - Updated POST/GET with new fields
-  `/app/api/bills/[id]/route.ts` - Updated PUT with new fields
-  `/app/api/bills/[id]/payment/route.ts` - **Added automatic date rolling**
-  `/app/api/ai/parse-command/route.ts` - NEW: LLM command parsing

Components

-  `/app/components/global-command-bar.tsx` - NEW: AI-powered quick add

-  `/app/app/layout.tsx` - Added GlobalCommandBar to layout

Documentation

-  `/CURRENT_STATE_ANALYSIS.md` - Codebase analysis
 -  `/DESIGN_DECISIONS.md` - All design decisions documented
 -  `/IMPLEMENTATION_PROGRESS.md` - Implementation status
 -  `/RUN_INSTRUCTIONS.md` - This file
-



UI Components Still Needed (Not Built Yet)

1. Update Bill Forms

- **File:** `/app/components/add-bill-dialog.tsx`
- **Add Fields:**
 - Provider (optional text input)
 - Policy Number (optional text input)
 - Reminder Days (text input, default “30,7,1”)

2. Update Bill Display

- **File:** `/app/components/bill-list.tsx`
- **Show:**
 - Provider name if available
 - Policy number if available
 - Reminder badges (30-day, 7-day, 1-day)
 - Overdue/due-soon indicators

3. Mark Paid Dialog

- **File:** `/app/bills/page.tsx`
- **Add Input:** Reference Number field when marking paid

4. Dashboard Widgets

- **Create:** `/app/components/dashboard-month-stats.tsx`
- Show income/expense/net for current calendar month
- Color-coded indicators
- Comparison with previous month
- **Create:** `/app/components/upcoming-obligations-widget.tsx`
- List upcoming bills/renewals (30/60 days)
- Color-coded by status (overdue, due soon, upcoming)
- Reminder badges
- Quick action to mark paid

5. SMS Inbox UI

- **Create:** `/app/sms-inbox/page.tsx`
- **Features:**
 - Textarea for pasting multiple SMS

- Parse button
- Table showing parsed transactions
- Approve/Edit/Reject buttons per row
- Batch approve option

Deployment

Development

```
npm run dev
```

Production Build

```
npm run build  
npm run start
```

Docker Deployment

```
# From repository root  
docker-compose up -d
```

Environment Variables for Production

Update `.env.production` with:

```
DATABASE_URL="postgresql://user:password@production-host:5432/finplanner_prod"  
NEXTAUTH_URL="https://your-domain.com"  
NEXTAUTH_SECRET="super-secure-production-secret-key"  
NODE_ENV="production"
```

Performance Considerations

Database Indexes (Recommended)

```
-- Add these indexes for better query performance  
CREATE INDEX idx_bills_user_next_due ON bills(user_id, next_due_date);  
CREATE INDEX idx_bill_instances_bill_due ON bill_instances(bill_id, due_date);  
CREATE INDEX idx_transactions_user_date ON transactions(user_id, date);
```

Support

Common Questions

Q: What happens when I mark a ONE_TIME bill as paid?

A: The status changes to PAID, a payment record is created, but the due date does NOT roll forward (as it's a one-time obligation).

Q: Can I edit the next due date after it rolls automatically?

A: Yes, you can manually edit any bill's next due date in the Edit Bill form.

Q: What if the LLM incorrectly parses my command?

A: You can edit the parsed data before confirming, or click "Edit" to re-parse with a modified input.

Q: Are SMS messages stored?

A: Yes, raw SMS text is stored in the `transaction.rawMessage` field for traceability.

Q: Can I disable automatic transaction creation when marking paid?

A: Yes, set `createTransaction: false` in the API request body (requires custom client implementation).

Contact

For issues or questions:

- Check `/TROUBLESHOOTING.md`
 - Review `/DESIGN_DECISIONS.md` for implementation details
 - Check `/CURRENT_STATE_ANALYSIS.md` for codebase structure
-

Verification Checklist

Before considering the implementation complete, verify:

- ☐ Database migration runs successfully
- ☐ Seed data creates sample renewals
- ☐ Can create a renewal with provider/policy number
- ☐ Renewal appears in upcoming obligations
- ☐ Marking paid creates payment record
- ☐ **Next due date rolls forward automatically**
- ☐ Transaction is created when marking paid
- ☐ Command bar opens with Cmd/Ctrl + K
- ☐ Command bar can parse transactions
- ☐ Command bar can parse renewals
- ☐ Dashboard shows month-to-date stats (when UI built)
- ☐ Overdue/due-soon indicators show correctly
- ☐ Reminder badges appear based on due date
- ☐ All unit tests pass
- ☐ API endpoints return expected data

You're All Set!

FinPlanner is now enhanced with comprehensive renewals tracking, AI-powered quick add, and better SMS parsing capabilities. Explore the features and enjoy hassle-free financial management!

Happy Tracking! 